Kelbari

Internship Program · 2026

🔐 Security Analyst internship.

A guided, portfolio-building track. Pick your tasks, build real projects, and share your work. Top performers can earn a Letter of Recommendation and a pathway to a job at Kelbari. One connected team for web development, digital marketing and graphic design, based in Bahawalpur, Pakistan.

Beginner (Tasks 1–6) | Intermediate (Tasks 7–8) | Advanced (Tasks 9–10)

Track guide & task cards
kelbari.net

Apply via the form in our
Kelbari LinkedIn post

Before you start

This checklist applies to every intern on every track. Complete it before submitting any task.

Profile & setup (do this first)

01 Complete your LinkedIn profile. If you need help, search “LinkedIn profile optimization and resume building” on YouTube and watch a tutorial before starting any project work.

02 Search “how to build a personal portfolio” on YouTube and watch a walkthrough — a simple portfolio makes your internship work easy to showcase.

03 Create your GitHub repository. Name it exactly: KELBARI (all caps). This single repository holds all your task submissions across the internship. Do not create a separate repo per task.

04 Connect your GitHub account and make sure your profile is public so mentors can review your work.

Per-task workflow (repeat for every task)

01 Step 1 — Review. Read the task card for your chosen task carefully. Understand the full feature checklist before writing a single line of code or starting a design.

02 Step 2 — Build. Develop your project, following the tech stack and requirements in the task card.

03 Step 3 — Push to GitHub. Commit your completed project to your KELBARI repository using this folder format, with no exceptions: KELBARI/[Track]-[Level or Task]-[ProjectName]/ — for example KELBARI/WebDev-L1-LandingPage/ or KELBARI/Python-Task2-BMICalculator/. Each folder must contain your source code, a README.md describing the project, and any relevant screenshots or output files.

04 Step 4 — Record a demo video. Record a short screen walkthrough of your finished project. Begin with a 2-second title card showing (1) your full name, (2) your track, and (3) the task title. After the title card, show the project working end to end — not just screenshots. For report-based tasks, narrate your document on screen.

05 Step 5 — Post to LinkedIn. Share your demo video in a LinkedIn post. Tag Kelbari and include the hashtag #kelbari, plus relevant domain hashtags (for example #webdevelopment, #datascience, #python, #cybersecurity, #internship).

06 Step 6 — Engage with peers. Watch the LinkedIn demo videos of at least two other interns from your cohort and leave a thoughtful comment on each — engage with their work, ask a question, or offer a constructive observation. Generic comments like “Great work!” don't count.

07 Step 7 — Submit. Submit your task through the submission form shared with you by Kelbari, including your GitHub repository link.

08 Step 8 — Evaluation. After the submission deadline, your work is reviewed. On successful completion you receive your Kelbari Completion Certificate and, where applicable, a Letter of Recommendation.

Key reminders

01 GitHub repo name must be exactly KELBARI (all caps) — any other name may cause your submission to be missed.

02 Follow the strict folder format KELBARI/[Track]-[Level or Task]-[ProjectName]/ — inconsistent naming delays review.

03 The first 2 seconds of every demo video must show your full name, track and task title as a static frame.

04 Thoughtful comments on at least two fellow interns' videos are required for completion.

05 Check your track's completion rule below and don't under-submit.

06 All work must be entirely your own. Plagiarism or copied work leads to cancellation of your submission and certificate.

What you receive from Kelbari

01 Guidance and mentorship from the Kelbari team throughout the program.

02 A Certificate of Completion once you finish your track's requirements.

03 For top performers: a Letter of Recommendation, a letter of appreciation, and a pathway to a job or paid role at Kelbari and related opportunities.

🔐 Security Analyst

Completion rule. Complete at least 4 of the 10 tasks below. Tasks span three tiers: Beginner (1–6), Intermediate (7–8) and Advanced (9–10). Mix tiers freely. Practical tasks need a GitHub repo and a demo video; research tasks need a well-structured .md report (no video).

Please read. Every practical task is strictly defensive and educational, and must be done only in your own isolated lab — a local virtual machine or an intentionally vulnerable practice app you host yourself. Never scan, test or capture traffic on systems, networks or websites you do not own or have written permission to test. Each card below keeps the goal at the learning-and-documentation level; follow the official tool documentation for exact commands.

Beginner (Tasks 1–6)

BEGINNER (TASKS 1-6) · TASK 1

Network Scanning Basics with Nmap

Lab-only: Use only a local VM you set up yourself (e.g. a practice Linux VM in VirtualBox).

Objective. Learn how port scanning works by scanning a machine you own in your own lab, and document which services are exposed and why that matters.

`Tech: Nmap, a local VM`

FEATURE CHECKLIST

- [ ] Install Nmap and document the steps in README.md
- [ ] Run a basic host scan of your own lab VM and record the results
- [ ] Run a service-version scan and an OS-detection scan on the same VM
- [ ] List the open ports found and the service on each
- [ ] For each open port, briefly explain what the service does and whether it is a risk
- [ ] Save your findings in a structured results file and include terminal screenshots
- [ ] README.md explains what Nmap is, why scanning matters defensively, and the ethical rules

> **Where to learn:** Search “Nmap tutorial beginners” on YouTube. A free Kali Linux VM from kali.org ships with Nmap. Reference the official Nmap docs for flag meanings, and read about common open-port risks to write your analysis.

BEGINNER (TASKS 1-6) · TASK 2

Firewall Configuration with UFW

Lab-only: Use a local Ubuntu or Kali VM you control.

Objective. Set up and configure a basic firewall on a Linux VM using UFW, applying rules that allow and deny specific traffic.

`Tech: UFW, Linux (in a VM)`

FEATURE CHECKLIST

- [ ] Install and enable UFW
- [ ] Add a rule to allow SSH
- [ ] Add a rule to deny HTTP
- [ ] Add at least 2 more rules of your choice (e.g. allow HTTPS, deny an IP range)
- [ ] Verify active rules and screenshot the status
- [ ] Describe how you tested that denied traffic is blocked
- [ ] Provide a script that applies all your rules in sequence
- [ ] README.md explains what a firewall does, what each rule achieves, and why you chose them

> **Where to learn:** Search “UFW firewall Ubuntu tutorial” on YouTube. Reference the official UFW documentation for commands, and read about Linux firewall best practices to inform your rules.

BEGINNER (TASKS 1-6) · TASK 3

Web Vulnerability Demonstration (Local Practice App)

Lab-only: Use a deliberately vulnerable training app (such as DVWA) installed on your own machine, at its lowest security setting, for learning only. Never test input flaws against any real site.

Objective. Using an intentionally vulnerable practice application you host locally, demonstrate how an input-validation flaw can expose data, and — most importantly — document how a developer prevents it.

```
Tech: A local vulnerable training app, a web browser
```

FEATURE CHECKLIST

- [ ] Install and configure the practice app locally and document the setup
- [ ] Demonstrate, at a high level, how an unsanitised input field can return more data than intended
- [ ] Record what information was exposed in your local demo, with screenshots
- [ ] README.md explains in plain language what the vulnerability is, why unsanitised input is dangerous, and how a developer fixes it with parameterised queries / prepared statements
- [ ] Include an accompanying notes file summarising your observations and the fix

> **Where to learn:** Reference the PortSwigger Web Security Academy (portswigger.net/web-security) for a thorough, defence-focused explanation of this vulnerability class and how prepared statements prevent it — the best free resource. Search “DVWA setup tutorial” for the local lab setup.

BEGINNER (TASKS 1-6) · TASK 4

Research Report: Common Network Security Threats

Objective. Write a well-structured research report on common network security threats — how each works, its real-world impact, and specific mitigations.

`Deliverable: Markdown report`

FEATURE CHECKLIST

- [ ] Report saved as network_security_threats_report.md
- [ ] Introduction: why network threats matter today
- [ ] DoS/DDoS: how it works, a real incident, impact, 3 mitigations
- [ ] Man-in-the-Middle: explanation, example, impact, 3 mitigations
- [ ] IP Spoofing: explanation, example, impact, 3 mitigations
- [ ] DNS Poisoning (bonus 4th threat): explanation, example, impact, 3 mitigations
- [ ] Comparison table across attack vector, who's at risk, difficulty and ease of mitigation
- [ ] Conclusion: 3 takeaways for a network administrator
- [ ] References: at least 4 credible sources

> **Where to learn:** Use NIST (nist.gov), CISA (cisa.gov) and the SANS Reading Room as primary sources. Reference MITRE ATT&CK for technical descriptions, and reputable security publications for real-world examples.

BEGINNER (TASKS 1-6) · TASK 5

Research Report: Social Engineering Attacks

Objective. Research and write a report on social engineering — focusing on phishing, pretexting and baiting — with case studies and prevention recommendations.

```
Deliverable: Markdown report
```

FEATURE CHECKLIST

- [ ] Report saved as social_engineering_report.md
- [ ] Introduction: define social engineering and why it is so effective (cite statistics if available)
- [ ] Phishing: types (spear, whaling, vishing, smishing), how it works, a case study, 4 preventions
- [ ] Pretexting: definition, how a false scenario is built, a case study, 3 preventions
- [ ] Baiting: physical and digital, a case study, 3 preventions
- [ ] Quid pro quo (bonus): explanation and prevention
- [ ] Comparison table: attack type, target, psychological lever, best countermeasure
- [ ] A 5-point employee awareness-training checklist
- [ ] References: at least 4 credible sources

> **Where to learn:** Use the SANS Reading Room for academic-quality papers and the CISA social-engineering guidance for official material. Source case studies from reputable security journalism.

BEGINNER (TASKS 1-6) · TASK 6

Research Report: The Importance of Patch Management

Objective. Explain what patch management is, why unpatched systems are one of the largest attack surfaces, and how organisations should patch effectively.

`Deliverable: Markdown report`

FEATURE CHECKLIST

- [ ] Report saved as patch_management_report.md
- [ ] Introduction: define patch management and its role in the vulnerability lifecycle
- [ ] Why patches matter: how vulnerabilities are found, reported (CVEs) and exploited; at least 2 real breaches caused by unpatched systems
- [ ] Consequences of not patching: breaches, ransomware, compliance and financial penalties (cite statistics)
- [ ] The lifecycle: Discovery → Assessment → Testing → Deployment → Verification
- [ ] Best practices: a prioritised 7-step checklist
- [ ] Challenges and how to overcome them
- [ ] References: at least 4 credible sources

> **Where to learn:** Reference NIST SP 800-40 (the definitive free guide), the CVE database and the CVSS scoring system. Use well-documented historical incidents as examples.

Intermediate (Tasks 7–8)

INTERMEDIATE (TASKS 7-8) · TASK 1

Vulnerability Scanning with Nikto

Lab-only: Scan only a local test server you control (your local practice app or a simple web server in a VM).

Objective. Run an automated web-vulnerability scan against a test server you host locally, analyse the results, and document the issues with remediation steps.

```
Tech: Nikto, a local test web server
```

FEATURE CHECKLIST

- [ ] Install Nikto and document the process
- [ ] Run a basic Nikto scan against your local target and save the output to a file
- [ ] List the issues reported
- [ ] For each finding, write what it is, why it is a risk, and how to fix it
- [ ] Categorise findings by severity (High/Medium/Low/Info)
- [ ] Run a second scan with the SSL flag if your target supports HTTPS
- [ ] Include screenshots of the scan and output
- [ ] README.md explains what Nikto does, its limitations (it is a “noisy” scanner — explain this), and how it differs from Nmap

> **Where to learn:** Search “Nikto web vulnerability scanner tutorial” on YouTube. Use your local practice app as the target. Reference the Nikto GitHub repo, and cross-reference findings against the OWASP Top 10 (owasp.org).

INTERMEDIATE (TASKS 7-8) · TASK 2

Capture Network Traffic with Wireshark

> **Lab-only:** Capture only on a network or VM you own and administer — never on public, university or shared Wi-Fi.

Objective. Capture live traffic on your own network interface, filter by protocol, analyse packets, and document security observations.

```
Tech: Wireshark, a local interface you control
```

FEATURE CHECKLIST

- [ ] Install Wireshark and note any permissions required
- [ ] Capture at least 2 minutes of traffic on your own interface
- [ ] Filter for HTTP traffic and screenshot the results
- [ ] Filter for DNS traffic and screenshot the results
- [ ] Filter for TCP and analyse a complete 3-way handshake (annotate SYN, SYN-ACK, ACK)
- [ ] Export the capture file and include it in the repo
- [ ] Identify one packet containing unencrypted data and note what is visible
- [ ] README.md explains why unencrypted HTTP is dangerous and how HTTPS prevents eavesdropping
- [ ] Include a glossary: packet, protocol, port, payload, handshake

> **Where to learn:** Search “Wireshark tutorial beginners” on YouTube. Reference the Wireshark User Guide for filter syntax, and search specifically for the TCP three-way handshake and HTTP packet analysis.

Advanced (Tasks 9–10)

ADVANCED (TASKS 9-10) · TASK 1

Secure-Coding Deep Dive: Preventing Injection (Report + Lab)

> **Lab-only:** All hands-on work stays inside your own local practice app. The emphasis and the graded deliverable are the defensive report, not offensive technique.

Objective. Building on the beginner web-vulnerability task, go deeper on the defensive side: study how injection flaws arise, confirm the behaviour once in your own local lab, and produce a thorough remediation report aimed at developers and managers.

```
Tech: A local vulnerable training app, Markdown for the report
```

FEATURE CHECKLIST

- [ ] Set up the practice app and confirm the vulnerable behaviour once, locally
- [ ] Keep a short, factual log of what you observed (no attack toolkit required)
- [ ] Remediation section: explain (1) why the vulnerability exists and (2) how to prevent it with parameterised queries / prepared statements, with a code example in two languages
- [ ] A developer checklist for writing injection-safe data access
- [ ] README.md: an executive summary for a non-technical manager describing the risk and business impact

> **Where to learn:** Reference the OWASP SQL Injection Prevention Cheat Sheet (cheatsheetseries.owasp.org) and the PortSwigger Web Security Academy for the definitive defensive guidance. Search “parameterised queries prevent SQL injection” for the remediation examples.

ADVANCED (TASKS 9-10) · TASK 2

Full Security Assessment Report (Your Own Lab)

Lab-only: Scope is strictly your own lab network/VMs. Define and record that scope in writing before you begin.

Objective. Run a structured, end-to-end security assessment of a small test network you own, using the tools from earlier tasks, and produce a professional report suitable for a technical team or management.

```
Tech: Nmap, Wireshark, Nikto (where applicable), Markdown
```

FEATURE CHECKLIST

- [ ] Define the assessment scope in writing: which lab IPs, which services, which time window
- [ ] Phase 1 — Reconnaissance: Nmap service/OS scan of your lab; document hosts, ports and services
- [ ] Phase 2 — Traffic analysis: capture 5+ minutes on your lab interface; analyse HTTP, DNS and ARP; note any unencrypted sensitive data
- [ ] Phase 3 — Web scan (if a web server is present): run Nikto against your lab server
- [ ] Findings register: a table with Finding ID, Description, Severity, Affected Asset, Recommended Fix
- [ ] Executive summary: a 1-page non-technical risk overview
- [ ] Technical report: detailed findings per phase with screenshots
- [ ] Remediation roadmap: prioritised fixes with effort estimates (Easy/Medium/Hard)
- [ ] Commit the report and evidence files to GitHub

> **Where to learn:** Reference the OWASP Web Security Testing Guide and the PTES technical guidelines for structure, and the CVSS scoring system (first.org/cvss) for consistent severity ratings. Search “how to write a penetration test report” for professional report format.

Contact & support

Questions during your internship? Reach the Kelbari team here.

Email	kelbariofficial@gmail.com
Support	support.kelbari@gmail.com
WhatsApp	+92 301 9233246 (available 24/7)
Website	kelbari.net
LinkedIn	Follow the Kelbari company page on LinkedIn

The purpose of this internship is to learn and grow. Treat every task with professional care, ask for guidance when you need it, and use the work to strengthen your portfolio and

confidence. **— The Kelbari Team**